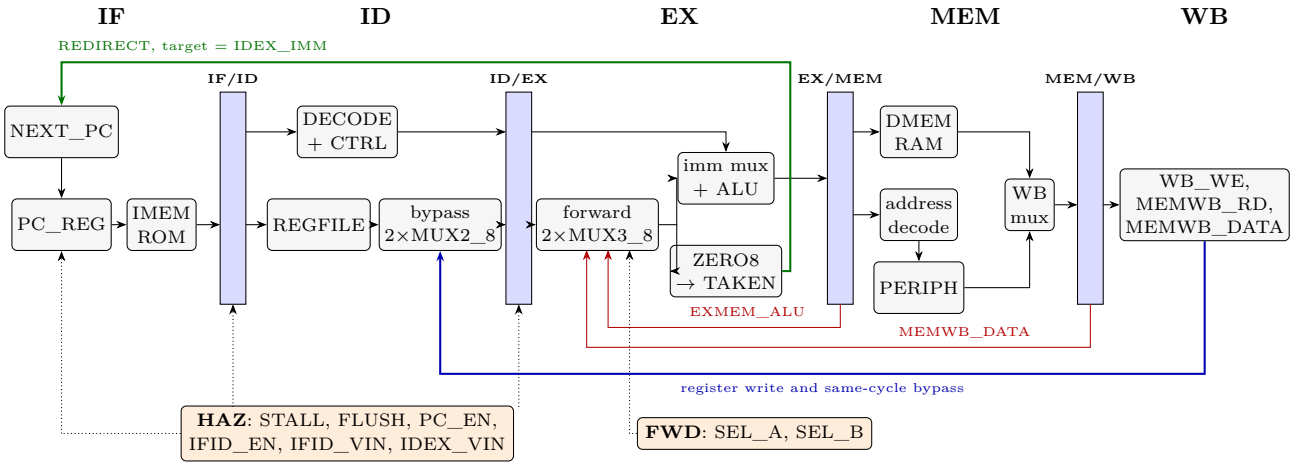
Architecture

ELL7282 Assignment 2 · Dharun Thota (Entry No. 2026EET2401)

1 Datapath



Single-issue, in-order, five stages, one global rising-edge clock.

- **main** (top-level circuit): the Clock, the RESET and plant input pins, every output pin, `NEXT_PC`, PC_REG, IMEM, the four pipeline registers, `ID_STAGE` (DECODE, CTRL, REGFILE, bypass), `EX_STAGE` (forwarding muxes, ALU, branch test), `MEM_STAGE` (RAM, address decode), the hazard unit `HAZ`, the forwarding unit `FWD` and `PERIPH`.
- Every mux, decoder, comparator, adder and register-file select is built from gates; the built-in parts are ROM, RAM, Register, gates, Splitter, Tunnel, Pin, Constant and Clock.

2 Pipeline registers

Each entry carries a valid bit V; V = 0 is a bubble. A field `F` of register `R` is called `R_F`. The PC copies are for debugging only.

Register	Load	Fields (bits)
IF/ID (25)	EN = ¬STALL	V, PC (8), INSTR (16)
ID/EX (53)	every edge	V, PC (8), isRW, isImm, isLD, isST, isBEQZ, isBNEZ, isHALT, USE_A, USE_B, aluSignals (5: pass, add, sub, and, sltu), RD (3, the [10:8] field: destination and first source), RB (3), A (8), B (8), IMM (8)
EX/MEM (31)	every edge	V, PC (8), isRW, isLD, isST, RD (3), ALU (8: result or address), SD (8: forwarded store data)
MEM/WB (21)	every edge	V, PC (8), isRW, RD (3), DATA (8: load data or ALU result, selected in MEM)

Next valid bits, loaded at every edge:

$$\begin{aligned} \text{IFID_V}' &= \neg(\text{FLUSH} \vee \text{HALTED}) \quad \text{(only when IFID_EN} = 1\text{)} \\ \text{IDEX_V}' &= \text{IFID_V} \wedge \neg\text{STALL} \wedge \neg\text{FLUSH} \\ \text{EXMEM_V}' &= \text{IDEX_V} \\ \text{MEMWB_V}' &= \text{EXMEM_V} \end{aligned}$$

A bubble keeps its leftover fields; only V makes it harmless, so every architectural effect is ANDed with V (section 6).

3 Stage logic and forwarding

Stage	Function
IF	PC addresses IMEM. PC ← REDIRECT ? IDEX_IMM : PC + 1 when PC_EN = 1.
ID	DECODE and CTRL give the control word and USE_A, USE_B. REGFILE reads R[RA], R[RB]; a register written in WB this cycle is bypassed to the read.
EX	Forwarding muxes pick each operand, then the immediate mux, the ALU and the branch test. The store data SD is the forwarded A.
MEM	RAM or peripheral access chosen by the address; reserved addresses read 0. DATA = load data or ALU result.
WB	R[MEMWB_RD] ← MEMWB_DATA at the edge, when WB_WE = 1.

Forwarding, for operand A (B is the same with IDEX_RB):

$$\begin{aligned} h_M &= \text{EXMEM_V} \wedge \text{EXMEM_isRW} \wedge (\text{EXMEM_RD} = \text{IDEX_RD}) \\ h_W &= \text{MEMWB_V} \wedge \text{MEMWB_isRW} \wedge (\text{MEMWB_RD} = \text{IDEX_RD}) \\ A_F &= h_M \ ?\ \text{EXMEM_ALU} \ :\ \big(h_W \ ?\ \text{MEMWB_DATA} \ :\ \text{IDEX_A}\big) \end{aligned}$$

EX/MEM, the youngest producer, has priority. The bypass in ID uses the same test with WB_WE = MEMWB_V ∧ MEMWB_isRW. A load in EX/MEM is never forwarded to a used operand, because the load-use stall comes first.

4 Stall and flush (unit HAZ)

$$\begin{aligned} \text{REDIRECT} &= \text{IDEX_V} \wedge \text{TAKEN} \\ \text{HALT_NOW} &= \text{IDEX_V} \wedge \text{IDEX_isHALT} \\ \text{FLUSH} &= \text{REDIRECT} \vee \text{HALT_NOW} \\ \text{STALL} &= \text{IFID_V} \wedge \text{IDEX_V} \wedge \text{IDEX_isLD} \wedge \text{USES_LOAD} \wedge \neg\text{FLUSH} \\ \text{USES_LOAD} &= (\text{USE_A} \wedge \text{RA} = \text{IDEX_RD}) \vee (\text{USE_B} \wedge \text{RB} = \text{IDEX_RD}) \\ \text{PC_EN} &= \text{REDIRECT} \vee \neg(\text{STALL} \vee \text{HALT_NOW} \vee \text{HALTED}) \\ \text{IFID_EN} &= \neg\text{STALL} \end{aligned}$$

- **Load-use:** one bubble. PC and IF/ID hold, ID/EX gets V = 0, the load moves on and is forwarded from MEM/WB a cycle later.
- **Taken branch or jump** (resolved in EX): IF/ID and ID/EX get V = 0, so the two wrong-path instructions die; the PC loads the target.
- **Source-use detection:** USE_A and USE_B come from the opcode and I, so immediates, targets and destinations never cause a false stall (pipeline_traces.pdf).

- **Priority:** ¬FLUSH in STALL lets a redirect override a younger stall.

5 Reset and HALT

RESET is asynchronous on every register: PC = 0, HALTED = 0 and all four valid bits = 0; PERIPH clears ACT, DISPLAY, the timer, START_PEND, the START edge detector and TRIP. REGFILE and RAM are not reset, so programs initialise them. A valid HALT in EX sets HALT_NOW: HALTED ← 1 at that edge, PC holds, IF/ID and ID/EX are invalidated, and the older instructions in MEM and WB complete normally. From then on IFID_V stays 0 until reset. A wrong-path HALT is killed by the flush before it reaches EX, so it never halts the machine. The timer and the safety logic keep running while halted.

6 Where the valid bit is used

- Register write: WB_WE = MEMWB_V ∧ MEMWB_isRW (also gates the bypass).
- RAM and peripheral writes: ST_WE = EXMEM_V ∧ EXMEM_isST.
- Branch, jump and HALT: IDEX_V in REDIRECT and HALT_NOW.
- Load-use stall: IFID_V and IDEX_V. Forwarding matches: EXMEM_V, MEMWB_V.
- Next ID/EX valid: IFID_V ∧ ¬STALL ∧ ¬FLUSH.

Because EX/MEM loads every cycle and memory never stalls, a store sits in MEM for exactly one cycle and commits at most once; a flushed or bubbled store has V = 0 and never commits.

7 Safety interlock (in PERIPH)

$$
\begin{aligned}
\text{MOTOR_EN_ACTUAL} &= \text{ACT}[2] \wedge \text{DOOR_CLOSED} \wedge \neg\text{TRIP} \\
\text{TRIP}' &= \big(\text{TRIP} \vee (\neg\text{DOOR_CLOSED} \wedge \text{ACT}[2])\big) \wedge \neg\text{REARM} \\
\text{RISE} &= \text{START} \wedge \neg\text{START_PREV} \\
\text{START_PEND}' &= \text{RISE} \vee (\text{START_PEND} \wedge \neg\text{ACK}) \\
\text{REARM} &= \text{ACK} \wedge \text{START_PEND} \wedge \text{DOOR_CLOSED} \wedge \neg\text{ACT}[2]
\end{aligned}
$$

ACK is a valid store with bit 0 = 1 to SYS_CTRL (0xF6). TRIP, START_PEND and START_PREV are REG1 flip-flops on the global rising-edge clock, cleared by RESET; MOTOR_EN_ACTUAL is plain gates.

Immediate disable. Opening the door drops MOTOR_EN_ACTUAL at once through the AND gate, with no clock edge and no CPU involvement.

Latch clocking. TRIP samples the door at the next rising edge (inputs are held for at least one clock), so it remembers the event; when the door closes again, TRIP still blocks the motor. One clock domain means no asynchronous set or clear races.

Safe rearm. Rearming needs a fresh low-to-high START edge (START_PEND), the door closed, the software request already cleared, and a software acknowledge. Software alone cannot create the START edge, and a cleared door sensor alone removes only one AND term.

Isolation from the pipeline. The pipeline reaches this logic only through the valid-qualified ACT write, so a flushed instruction or a bubble cannot affect the motor. MOTOR_EN_REQ (the latch bit) and MOTOR_EN_ACTUAL are separate output pins.